

Vendor Discovery & Outreach Agent

A plain-English guide to how the system finds vendors, writes call scripts, and runs AI-powered phone outreach

Built for the CALL-E Hackathon | Powered by CALL-E + OpenStreetMap + Groq AI

What Is This?

The Vendor Discovery & Outreach Agent is a software pipeline that does in minutes what normally takes a sourcing team days: it **finds local businesses** that could supply a product, **calls them automatically** to qualify their interest, and **collects pricing and availability details** in a second round — all without a human picking up the phone. Every vendor, every call, and every extracted data point is saved in a database and shown on a live web dashboard.

The One-Sentence Summary

You type a product name and a city. The system finds up to 20 vendors on OpenStreetMap, calls each one using an AI voice agent, and returns a structured table of who is interested, what they can supply, and at what price.

The Problem It Solves

Without this system	With this system
Manually searching maps and directories for every vendor	OSM query returns a ranked list with phones in seconds
Copy-pasting phone numbers one by one	Phones validated and formatted to E.164 automatically
Calling 20 shops yourself to find 2 that are interested	CALL-E AI agent calls all vendors and records outcomes
Typing up call notes after every conversation	Groq AI reads the transcript and extracts structured data
Losing track of who said what across 10 browser tabs	Everything in one database, visible on a live dashboard

The Three Core Modules

The system is split into three clearly separated modules. Each does one job, and they connect in a straight pipeline: discover -> script -> call.

1	Module 1 — Vendor Discovery (discovery.py) Queries OpenStreetMap's global open database to find businesses matching your product keyword in any city. Filters to only those with verified phone numbers. Returns name, phone (E.164 format), category, GPS coordinates, and stable OSM ID.
2	Module 2 — Script Generation (script_gen.py) Takes the product description and generates a natural-language goal script for the CALL-E AI agent. Round 1 qualifies interest; Round 2 collects pricing, quantities, timelines, and contact names. The client must approve before any call.
3	Module 3 — Call Pipeline (caller.py) Sends each vendor's phone and the approved script to CALL-E, which plans the call, dials, and returns a full transcript. The pipeline polls for completion, then runs Groq AI on the transcript to extract structured intelligence.

Module 1 — Vendor Discovery in Detail

Source: OpenStreetMap (OSM) — the world's largest free geographic database, maintained by millions of contributors and licensed under ODbL.

How it works: The system sends a structured query to the Overpass API asking: *'Give me all businesses tagged as a shoe shop / hardware store / etc. that also have a phone number, within a bounding box around this city.'* The bounding box is automatically calculated from the city name using the Nominatim geocoder.

What the query returns per vendor:

- **name** — Business name as listed in OSM
- **phone** — Reformatted to E.164 international format (+441234567890)
- **masked_phone** — Masked version for safe display (+44****890)
- **category** — OSM tag matched (e.g. shop=shoes, shop=hardware)
- **lat / lon** — GPS coordinates, used for timezone-aware business hours check
- **candidate_id** — Stable URI in the form osm:<id> for audit trails

Business Hours Gate

Before placing any call, the system resolves the vendor's local timezone using GPS coordinates. Calls outside Monday-Friday 09:00-18:00 local time are skipped automatically and logged as 'outside_business_hours'. This prevents calling a London shop at 3am or a Delhi vendor on a Sunday.

Module 2 — Script Generation in Detail

The AI phone agent needs clear instructions for what to say. These are called a **goal script**. Module 2 generates two scripts from a single product description:

Round 1 — Qualification Script

Introduces the caller as a sourcing agent. Asks: does the business sell this product? Would they be open to a follow-up about quantities and pricing? Keeps the call under 2 minutes. Ends politely regardless of the answer.

Round 2 — Data Capture Script

Used only with vendors who said yes in Round 1. Collects: contact name, supply capacity and quantities, rough price per unit, and earliest available timeline. Keeps the call under 5 minutes.

Client Approval Gate (Mandatory)

No call is ever placed without the client first seeing and approving the script. Interactively, the system shows the script and asks: Use as-is, Edit, or Quit. In automated mode (--yes), the approval timestamp is recorded in the database. The timestamp and consent basis are stored in every campaign record as a compliance audit trail.

Module 3 — The Call Pipeline in Detail

This module connects to CALL-E's API and manages the full lifecycle of each phone call — from planning through transcript extraction.

1	plan_call — Prepare the call
	Sends the phone number and goal script to CALL-E. The AI reads the script and prepares the voice agent. Returns a plan_id and confirm_token. No phone has rung yet.
2	run_call — Execute the call
	Sends the plan_id and confirm_token to actually dial. CALL-E's agent speaks naturally with whoever answers. Returns a run_id immediately (call is async).
3	poll_until_done — Wait for completion
	Every 15 seconds, checks CALL-E for current status. Terminal statuses: COMPLETED, FAILED, NO ANSWER, BUSY, DECLINED, CANCELLED. Space vs underscore variants are both handled. Times out after 10 minutes maximum.
4	Groq AI inference — Extract intelligence
	Reads the call transcript. Parses it into timestamped speaker lines. Sends to Groq Llama 3.3-70B to extract: interest level, sentiment, key signals, Round 2 recommendation, and a one-sentence summary. All stored in SQLite.
5	classify_round1 — Make the call/no-call decision
	Checks CALL-E's outcome.task_completed boolean. Falls back to transcript keyword scan if unavailable. Returns: positive negative no_answer failed. Only 'positive' leads proceed to Round 2.

What You Get Back

Output	Where	Contents
Terminal table	Console	Name, masked phone, category, status, AI summary per vendor
SQLite database	vendor_discovery.db	Full raw call data, transcripts, extracted fields, consent timestamps
Web dashboard	http://localhost:5000	Live view, auto-refreshes every 8s, colour-coded statuses
Excel export	file.xlsx (optional)	One row per vendor: all R1+R2 fields, AI extractions, candidate IDs

AI Inference — What Groq Extracts

After every completed call, the transcript is sent to Groq's free API (Llama 3.3-70B, 14,400 requests/day free tier) for structured extraction.

Round	Field	What it means
R1	interest_level	high / medium / low / none / unknown
R1	sentiment	positive / neutral / negative overall tone
R1	key_signals	Exact phrases from the vendor indicating interest or refusal
R1	recommend_round2	true / false — whether to proceed to a follow-up call
R1	summary	One sentence describing the call outcome
R2	contact_name	Name of the right person to speak with
R2	can_supply	true / false / null — whether they can provide the product
R2	quantity_mentioned	Any quantity figure the vendor mentioned
R2	price_range	Price per unit or batch as stated by the vendor
R2	timeline	Earliest date or lead time they quoted
R2	next_steps	What the vendor suggested as next action

Safety, Privacy & Compliance

Feature	What it does	Why it matters
Phone masking	Phones shown as +44****890 everywhere except the DB	Prevents accidental number exposure in logs or screenshots
Consent gate	Client must approve vendor list; timestamp recorded	Provides audit trail proving outreach was authorised
Business hours gate	GPS+timezone check blocks calls outside 09:00-18:00	Prevents nuisance calls; respects vendor working hours
Dry-run mode	--dry-run plans calls but never dials any number	Safe for demos and testing without spending call credits
Bounded retries	No-answer leads retried at most once; no loops	Controls call spend; avoids harassing uninterested vendors
No secrets in files	API keys via environment variables only; never hardcoded	Prevents credentials leaking through version control
OSM attribution	ODbL credit shown in all outputs and the dashboard	Legally required by the ODbL licence

How to Run It

Command	What it does
<code>python main.py --product "shoes" --location "London" --dry-run</code>	Discover vendors + generate script. NO real calls placed. Safe for testing.
<code>python main.py --product "hardware" --location "Mumbai" --limit 5 --yes</code>	Full campaign: discover 5 vendors, auto-approve, call all, run AI inference.
<code>python main.py ... --export-excel results.xlsx</code>	Same as above, plus export a formatted Excel file when done.
<code>python main.py ... --skip-hours-gate</code>	Bypass the business-hours check (for testing at weekends or off-hours).
<code>python dashboard.py</code>	Start the web dashboard at <code>http://127.0.0.1:5000</code> (auto-refreshes every 8s).

Technology Stack

Component	Technology	Role
Vendor discovery	OpenStreetMap / Overpass API	Free global business directory with GPS data
AI phone agent	CALL-E (calle.ai)	Plans, executes, and transcribes phone calls
Transcript AI	Groq API — Llama 3.3-70B	Free tier: 14,400 req/day, structured extraction
Data storage	SQLite	Zero-config local database, no server needed
Web dashboard	Flask + vanilla JavaScript	Lightweight, no build step, auto-refreshing UI
Business hours	timezonefinder + zoneinfo	GPS-to-timezone lookup, Python standard library
Excel export	openpyxl	Formatted .xlsx with colour-coded headers
Compliance agents	Claude Opus 4.8 subagents	5 specialised reviewers for consent, schema, security

Data (c) OpenStreetMap contributors, ODbL licence — openstreetmap.org/copyright | Built for CALL-E Hackathon, AIRUDDER Pte Ltd | Deadline: September 14 2026